

Module 8

JavaScript JavaScript Introduction

Que 1. What is JavaScript? Explain the role of JavaScript in web development.

Ans:- JavaScript is a high-level, interpreted programming language used to create interactive and dynamic web pages. It runs directly in the browser and is one of the core technologies of web development alongside HTML and CSS.

- HTML → Structure (skeleton)
- CSS → Styling (design)
- JavaScript → Behavior (interactivity)

Role of JavaScript in Web Development

1. Adds Interactivity

- Handles user actions like clicks, typing, scrolling
- Example: Button click → show message

2. DOM Manipulation

- DOM = Document Object Model (structure of webpage)
- JavaScript can **change content, styles, and elements dynamically**

3. Form Validation

- Checks user input before submitting forms
- Example:
 - Email format check
 - Required fields validation

4. Dynamic Content Loading

- Updates content without refreshing the page
- Used in modern apps like dashboards, social media

5. Event Handling

- Responds to user actions (events)
 - click
 - hover
 - keypress

Que 2. How is JavaScript different from other programming languages like Python or Java?

Ans:-

1. Execution Environment

JavaScript

- Runs directly inside the browser (Chrome, Edge, etc.)
- No installation needed for basic use
- Can also run on the server using Node.js

Python

- Runs on a Python interpreter installed on your system
- Mainly used for backend, scripting, and data processing
- Not used directly inside browsers

Java

- Runs on JVM (Java Virtual Machine)
- First compiled into bytecode, then executed
- Platform-independent ("Write Once, Run Anywhere")

2. Compilation VS Interpretation

JavaScript

- Interpreted language (modern engines use JIT – Just-In-Time compilation)
- Code runs immediately in browser

Python

- Fully interpreted
- Slower compared to compiled languages

Java

- Compiled + interpreted
- First compiled → then executed by JVM
- More efficient for large systems

Que 3. Discuss the use of <script> tag in HTML. How can you link an external JavaScript file to an HTML document?

Ans:- The <script> tag in HTML is used to **add JavaScript code** to a webpage. It allows you to make your website **interactive, dynamic, and functional**.

Why <script> Tag is Used

- Add interactivity (click events, animations)
- Validate forms
- Manipulate HTML elements (DOM)
- Load dynamic content
- Communicate with servers (APIs)

Ways to Use <script> Tag

1. Internal JavaScript (Inside HTML)

You can write JavaScript directly inside the <script> tag.

Example :-

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JS</title>
</head>
<body>

  <h2 id="demo">Hello</h2>

  <script>
    document.getElementById("demo").innerHTML = "Hello JavaScript!";
  </script>

</body>
</html>
```

2. External JavaScript

You can link a separate .js file using the src attribute.

Example:-

```
<!DOCTYPE html>
<html>
<head>
  <title>External JS</title>
```

```
</head>
<body>

<h2 id="demo">Hello</h2>

<!-- Linking JS file -->
<script src="script.js"></script>

</body>
</html>
```

Variables and Data Types Theory Assignment

Que 1. What are variables in JavaScript? How do you declare a variable using var, let, and const?

Ans:- A **variable** in JavaScript is a **container used to store data values**.

It allows you to save information (like numbers, text, or objects) and reuse or modify it later in your program.

Example Of a Variable

```
let name = "Krunal";
let age = 21;
```

How to Declare Variables in JavaScript

JavaScript provides **three keywords** to declare variables:

var, let, and const

1. Var

```
var x = 10;
```

Features:

- Function-scoped
- Can be **redeclared**
- Can be **updated**

```
var x = 10;
```

```
var x = 20; // ☒ allowed
```

```
x = 30;    // ☒ allowed
```

2. Let (Modern & Recommended)

```
let y = 15;
```

Features:

- Block-scoped (works inside {} only)
- Can be **updated**
- Cannot be redeclared in same scope

```
let y = 10;  
y = 20;    // ✓ allowed  
// let y = 30; ✗ error
```

3. Const (Constant Value)

```
const z = 50;
```

Features:

- Block-scoped
- Cannot be **updated**
- Cannot be **redeclared**

```
const z = 10;  
// z = 20; ✗ error
```

Que 2. Explain the different data types in JavaScript. Provide examples for each.

Ans:- In JavaScript, **data types** define the kind of value a variable can hold. They are mainly divided into **two categories**:

1. Primitive Data Types

These are **basic, single values** (immutable).

I. Number

Represents **numeric values** (integers & decimals)

```
let age = 25;  
let price = 99.99;
```

II. String

Represents **text data**

```
let name = "Krunal";  
let message = 'Hello World';
```

III. Boolean

Represents **true or false**

```
let isLoggedIn = true;  
let isAdmin = false;
```

IV. Undefined

A variable declared but **not assigned a value**

```
let x;  
console.log(x); // undefined
```

V. Null

Represents **intentional empty value**

```
let data = null;
```

👉 **Difference:**

- undefined → not assigned
- null → intentionally empty

Ques 3. What is the difference between undefined and null in JavaScript?

Ans:-

undefined	Null
Indicates a variable that has been declared but not yet assigned a value.	Represents the intentional absence of any object value.
Primitive data type in JavaScript.	Primitive data type in JavaScript.
Automatically assigned to variables that are declared but not initialized.	Must be explicitly assigned to a variable.
<code>undefined == null // true</code> (loose equality considers them equal).	<code>undefined == null // true</code> (loose equality considers them equal).
<code>undefined === null // false</code> (they are not strictly equal in type).	<code>null === undefined // false</code> (they are not strictly equal in type).
Used by JavaScript when a variable is not assigned a value.	Used intentionally by developers to indicate "no value" or "empty".
<code>undefined</code> is a global property with the value <code>undefined</code> .	<code>null</code> is a global property with the value <code>null</code> .
<code>undefined</code> values are omitted during serialization.	<code>null</code> values are preserved during JSON serialization (e.g., <code>{"key": null}</code>).

JavaScript Operators

Que.1 : What are the different types of operators in JavaScript? Explain with examples.

- **Arithmetic operators**
- **Assignment operators**
- **Comparison operators**
- **Logical operators**

Ans:- JavaScript provides different types of operators to perform operations on variables and values.

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Description	Example
+	Addition	$5 + 2 = 7$
-	Subtraction	$5 - 2 = 3$
*	Multiplication	$5 * 2 = 10$
/	Division	$5 / 2 = 2.5$
%	Modulus (Remainder)	$5 \% 2 = 1$
++	Increment	a++
--	Decrement	a--

2. Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Example	Meaning
=	x = 5	Assign value
+=	x += 2	$x = x + 2$

Operator	Example	Meaning
<code>-=</code>	<code>x -= 2</code>	<code>x = x - 2</code>
<code>*=</code>	<code>x *= 2</code>	<code>x = x * 2</code>
<code>/=</code>	<code>x /= 2</code>	<code>x = x / 2</code>

3. Comparison Operators

Comparison operators compare two values and return true or false.

Operator	Description
<code>==</code>	Equal to
<code>===</code>	Strict equal to
<code>!=</code>	Not equal to
<code>!==</code>	Strict not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

4. Logical Operators

Logical operators are used to combine multiple conditions.

Operator	Description
<code>&&</code>	Logical AND
<code>^</code>	
<code>!</code>	Logical NOT

Que.2 : What is the difference between == and === in JavaScript?

Ans:-

Operator	Name	Checks
==	Equality Operator	Compares only values
===	Strict Equality Operator	Compares both value and datatype

Control Flow (If-Else, Switch) Theory Assignment

Question 1: What is control flow in JavaScript? Explain how if-else statements work with an example.

Ans:- Control flow in JavaScript refers to the order in which statements and instructions are executed in a program. It helps in decision-making and controlling program execution.

The if-else statement is used to execute different blocks of code based on a condition.

Syntax:-

```
if (condition) {  
    // code if condition is true  
} else {  
    // code if condition is false  
}
```

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

Ans:- A switch statement is used to execute one block of code from multiple options.

Syntax:-

```
switch(expression) {  
    case value1:  
        // code  
        break;  
  
    case value2:  
        // code  
        break;  
  
    default:  
        // default code  
}
```

Loops (For, While, Do-While)

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Ans:- Loops are used to repeat code multiple times.

1. for Loop

Used when the number of iterations is known.

Example

```
for(let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

2. while Loop

Executes while the condition is true.

Example

```
let i = 1;  
  
while(i <= 5) {  
  console.log(i);  
  i++;  
}
```

3. do-while Loop

Executes code at least once before checking the condition.

Example

```
let i = 1;  
  
do {  
  console.log(i);  
  i++;  
} while(i <= 5);
```

Question 2: What is the difference between a while loop and a do-while loop?

Ans:-

while Loop	do-while Loop
Checks condition before execution	Checks condition after execution
May not execute even once	Executes at least once

Functions

Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.

Ans:- Functions are reusable blocks of code designed to perform specific tasks.

Function Declaration Syntax

```
function functionName() {  
    // code  
}
```

Calling a Function

```
function greet() {  
    console.log("Hello");  
}
```

Question 2: What is the difference between a function declaration and a function expression?

Ans:-

Function Declaration	Function Expression
Declared using function keyword	Stored inside a variable
Can be called before declaration	Cannot be called before declaration

Function Declaration

```
function add(a, b) {  
  return a + b;  
}
```

Function Expression

```
const add = function(a, b) {  
  return a + b;  
};
```

Question 3: Discuss the concept of parameters and return values in functions.

Ans:-

Parameters

Parameters are variables passed into a function.

Return Value

A return value sends data back from the function.

Example

```
function multiply(a, b) {  
  return a * b;  
}
```

```
let result = multiply(4, 5);  
console.log(result);
```

Arrays

Question 1: What is an array in JavaScript? How do you declare and initialize an array?

Ans:- An array stores multiple values in a single variable.

Declaration and Initialization

```
let fruits = ["Apple", "Banana", "Mango"];
```

Question 2: Explain the methods push(), pop(), shift(), and unshift() used in arrays.

Ans:-

Method	Description
push()	Adds element at end
pop()	Removes last element
shift()	Removes first element
unshift()	Adds element at beginning

Objects

Question 1: What is an object in JavaScript? How are objects different from arrays?

Ans:- Objects store data in key-value pairs.

Example

```
let student = {  
  name: "Rahul",  
  age: 20  
};
```

Difference Between Objects and Arrays

Object	Array
Stores key-value pairs	Stores ordered values
Uses property names	Uses index numbers

Question 2: Explain how to access and update object properties using dot notation and bracket notation.

Ans:- **Dot Notation**

```
let person = {  
  name: "Amit"  
};  
  
console.log(person.name);  
  
person.name = "Raj";
```

Bracket Notation

```
console.log(person["name"]);  
  
person["name"] = "Kunal";
```

JavaScript Events

Question 1: What are JavaScript events? Explain the role of event listeners.

Ans:- Events are actions that occur in the browser, like clicks or key presses.

Event Listeners

Event listeners detect and respond to events.

Question 2: How does the `addEventListener()` method work in JavaScript? Provide an example

Ans:- The `addEventListener()` method attaches an event handler to an element.

Example

```
<button id="btn">Click Me</button>  
  
<script>  
document.getElementById("btn").addEventListener("click", function() {  
  alert("Button Clicked!");  
});  
</script>
```

DOM Manipulation

Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?

Ans:- DOM stands for Document Object Model. It represents HTML elements as objects so JavaScript can access and modify them.

JavaScript interacts with the DOM to:

- Change content
- Change styles
- Add or remove elements

Question 2: Explain the methods getElementById(), getElementsByClassName(), and querySelector() used to select elements from the DOM.

Ans:- 1. getElementById()

Selects an element by ID.

```
document.getElementById("demo");
```

2. getElementsByClassName()

Selects elements by class name.

```
document.getElementsByClassName("box");
```

3. querySelector()

Selects the first matching CSS selector.

```
document.querySelector(".box");
```


JavaScript Timing Events (setTimeout, setInterval)

Question 1: Explain the setTimeout() and setInterval() functions in JavaScript. How are they used for timing events?

Ans:-

Function	Description
setTimeout()	Executes code once after delay
setInterval()	Repeats code after interval

Example

```
setInterval(() => {  
  console.log("Hello");  
}, 1000);
```

Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.

Ans:- `setTimeout(() => {
 console.log("This message appears after 2 seconds");
}, 2000);`

JavaScript Error Handling

Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.

Ans:- Error handling manages runtime errors to prevent program crashes.

try-catch-finally Syntax

```
try {  
  let result = 10 / 0;  
  console.log(result);  
}  
catch(error) {  
  console.log("An error occurred");  
}  
finally {
```

```
    console.log("Execution completed");  
}
```

Explanation

- try → contains risky code
- catch → handles errors
- finally → always executes

Question 2: Why is error handling important in JavaScript applications?

Ans:- Error handling is important because it:

- Prevents application crashes
- Improves user experience
- Helps debug problems
- Maintains smooth program execution

Example:

```
try {  
    console.log(data);  
}  
catch(error) {  
    console.log("Variable not found");  
}
```